PROJECT DOCUMENTATION

Databricks Asset Bundles (DAB)

CI/CD Deployment Automation with GitHub Actions

Automated DEV → TEST → PROD Deployment Pipeline for Databricks Jobs and Lakeflow Pipelines

Prepared: September 2026

Project Type: Proof of Concept (POC)

By: Manibabu Dnv - HRM6500

Table of Contents

1. Executive Summary

This document describes the end-to-end design and implementation of an automated deployment pipeline for a Databricks project. The solution uses Databricks Asset Bundles (DAB) to define Databricks resources — Jobs, Lakeflow Declarative Pipelines, and Python artifacts — as version-controlled code, and integrates this with GitHub Actions to automatically validate and promote the project through DEV, TEST, and PROD environments.

The final solution provides a pull-request-based validation gate, fully automated deployment to DEV and TEST on merge to the main branch, and a manual approval gate protecting PROD. Authentication to Databricks is handled securely through a dedicated service principal using OAuth Machine-to-Machine (M2M) credentials stored as GitHub Secrets, with no dependency on personal user accounts.

A key outcome of the project was proving that the CI/CD service principal does not require Databricks workspace administrator privileges to operate — the pipeline was successfully re-tested and confirmed working after admin access was revoked, satisfying the principle of least privilege.

2. Project Objective

The goal of the project was to implement deployment automation for a Databricks project across three environments — DEV, TEST, and PROD — removing the need to manually create or update Databricks resources through the web UI in each environment.

Target Workflow

```
Developer
   |
   v
GitHub  ->  Pull Request  ->  Validation  ->  Merge to main
   |
   v
Automatic deployment: DEV -> TEST -> PROD (approval gate)
```

Success criteria for the POC included: a working local DAB deployment, a fully automated GitHub Actions pipeline, secure non-interactive authentication, an approval control before production changes, and confirmation that the automation works without elevated (admin) permissions.

3. Technology Stack — Quick Reference

The table below is a fast lookup of every technology used in the project. Section 4 (Understanding the Key Components) explains each one in more depth, in plain language.

Technology	Role in the Project
Databricks Asset Bundles (DAB)	Defines Databricks Jobs, Pipelines, artifacts, and permissions as source-controlled YAML/code, and deploys them consistently across environments.
Databricks Jobs	Executes the data engineering workload (sample_job), using the built Python wheel.
Lakeflow Declarative Pipeline	Implements the ETL pipeline (DatabricksDAB_etl) that transforms data through the bronze/silver/gold layers.

Technology	Role in the Project
Python / pyproject.toml	Defines the Python project, its dependencies, and build configuration.
uv	Python packaging tool used to build the project into a distributable .whl file (uv build --wheel).
Git / GitHub	Version control and the source of truth for the DAB project (code, configuration, and history).
GitHub Actions	CI/CD automation engine that triggers on pull requests and pushes to main, and executes Databricks CLI commands.
Databricks Service Principal	A machine identity (dab-cicd-poc) used by GitHub Actions to authenticate to Databricks, instead of a personal account.
OAuth M2M	Machine-to-machine authentication mechanism using a client ID and client secret to authorize the service principal.
GitHub Secrets	Secure storage for DATABRICKS_HOST, DATABRICKS_CLIENT_ID, and DATABRICKS_CLIENT_SECRET, injected into the workflow at runtime.
GitHub Environments	Provides the manual approval gate that protects deployment to the PROD environment.

4. Understanding the Key Components

This section explains, in plain language, every tool and concept used in the project. It is meant to be read as a reference — if a term in a later section is unfamiliar, look it up here.

4.1 Databricks Asset Bundles (DAB)

Databricks Asset Bundles (DAB)

DAB is a framework from Databricks that lets you describe Databricks resources — Jobs, Lakeflow pipelines, notebooks, Python packages, and permissions — as code (YAML files) instead of clicking through the Databricks web UI. This is often called “infrastructure as code.” Once the project is defined as code, a single command deploys (creates or updates) the matching resources in a target workspace.

In this project: DAB was used to define the Job, the ETL Pipeline, the Python artifact, and the DEV/TEST/PROD configuration, and to deploy all of it consistently with one command: databricks bundle deploy.

> **In simple terms:**
> Instead of manually building Jobs and Pipelines by clicking around in Databricks, you write down what they should look like in a text file, and DAB builds them for you — the same way every time.

```
Project files  ->  databricks.yml  ->  DAB  ->  Databricks resources
```

4.2 Core Configuration Files

databricks.yml

The root configuration file of a bundle. It acts as the main entry point, declaring the bundle's name, which resource files to include, and how each environment (target) should be configured.

In this project: Declared the bundle name (DatabricksDAB), included every YAML file inside resources/, and defined the catalog/schema variables and the dev, test, and prod targets.

YAML

YAML ("YAML Ain't Markup Language") is a human-readable text format for configuration, using indentation instead of brackets or tags to show structure.

In this project: Every DAB file (databricks.yml, job and pipeline definitions) and every GitHub Actions workflow file is written in YAML.

pyproject.toml

A standard configuration file for Python projects that declares the project's name, dependencies, and how it should be built into a package.

In this project: Described the DatabricksDAB Python project so that uv knew how to build it into a wheel file.

4.3 Databricks Resources

Databricks Job

A Job is a unit of scheduled or on-demand work in Databricks that runs one or more tasks — for example, running a Python package, a notebook, or a SQL script — on a cluster.

In this project: The sample_job resource executes the workload, using the Python wheel that DAB builds and uploads during deployment.

Lakeflow Declarative Pipeline

A Databricks feature for building ETL (Extract, Transform, Load) data pipelines declaratively: you describe what the transformations should produce, and Databricks manages how the pipeline actually runs and orchestrates that work.

In this project: The DatabricksDAB_etl pipeline processes data through the transformation logic defined under src/DatabricksDAB_etl/transformations.

Catalog and Schema (Unity Catalog)

In Databricks' Unity Catalog, a catalog is a top-level container for data (similar to a database server), and a schema is a namespace inside it (similar to a database or folder of tables).

In this project: The catalog and schema variables controlled where each environment's pipeline data and objects were created — for example, schema: prod for the PROD target.

4.4 Python Packaging

Python wheel (.whl)

A wheel is the standard, pre-built package format for distributing Python code — conceptually similar to a zipped, installable package. It lets code be installed quickly without rebuilding it from source every time.

In this project: The project's Python source code was compiled into databricksdab-0.0.1-py3-none-any.whl before every deployment, so the Databricks Job could install and run it.

uv

uv is a fast, modern Python package manager and build tool (an alternative to pip and setuptools) used to install dependencies and build distributable packages.

In this project: The bundle ran uv build --wheel to produce the .whl artifact, both on the local machine and inside the GitHub Actions runner.

4.5 Source Control

Git

Git is a distributed version control system that records every change made to a set of files, allowing branching, merging, history review, and rollback.

In this project: Tracked every change to the DAB project's code and configuration, turning the project into a reviewable, versioned history rather than untracked manual edits.

GitHub

GitHub is a cloud platform for hosting Git repositories and collaborating on code, including pull requests, code review, and automation.

In this project: Hosted the DatabricksDAB repository, which became the single source of truth that both developers and the CI/CD pipeline worked from.

Pull Request (PR)

A request to merge changes from one branch into another (typically main), usually reviewed by a teammate before being accepted.

In this project: Opening a PR triggered bundle validation automatically, before any code was allowed to merge into main.

4.6 CI/CD Automation

CI/CD (Continuous Integration / Continuous Deployment)

A software engineering practice of automatically testing or validating changes (CI) and automatically releasing them to one or more environments (CD), instead of relying on manual, error-prone release steps.

In this project: CI = validating the bundle automatically on every pull request. CD = automatically deploying the bundle to DEV, then TEST, then (with approval) PROD after a merge to main.

GitHub Actions

GitHub's built-in automation engine. It runs scripted “workflows” made up of jobs and steps, triggered automatically by events such as a push or a pull request.

In this project: Executed the entire pipeline: installing the Databricks CLI and uv, then running databricks bundle validate and databricks bundle deploy for each environment.

Databricks CLI

A command-line tool for interacting with Databricks, including bundle commands such as validate, deploy, and run.

In this project: Installed on both the local development machine and the GitHub Actions runner to execute all bundle commands.

Bundle Target (DEV / TEST / PROD)

A named configuration profile inside a DAB bundle that supplies environment-specific values (such as catalog and schema names), so one codebase can be deployed differently to each environment.

In this project: Three targets — dev, test, and prod — were defined, each with its own variable values and deployment mode, selected at deploy time with the -t flag.

Deployment Mode (development vs. production)

A DAB setting that changes how resources are named and managed. Development mode isolates each developer's or environment's deployment to avoid naming collisions; production mode treats the deployment as the single, real, shared deployment.

In this project: The dev and test targets used development mode; the prod target used production mode, reflecting that PROD represents the real, shared production deployment.

4.7 Authentication and Security

Service Principal

A non-human, application-level identity used by automated systems to authenticate to a service, instead of relying on an individual person's login and credentials.

In this project: dab-cicd-poc is the service principal that GitHub Actions uses to authenticate to Databricks, keeping deployments independent of any one person's account.

OAuth M2M (Machine-to-Machine)

An authentication method where two systems exchange a client ID and client secret to obtain a temporary access token, with no human login step involved.

In this project: Authenticated the dab-cicd-poc service principal securely and non-interactively from inside the GitHub Actions workflow.

GitHub Secrets

Encrypted variables stored at the repository or organization level in GitHub, which can be referenced inside workflows without ever exposing their real values in the code.

In this project: Stored DATABRICKS_HOST, DATABRICKS_CLIENT_ID, and DATABRICKS_CLIENT_SECRET, injected into the workflow as environment variables at run time.

GitHub Environments

A GitHub feature for defining named deployment targets (such as prod) with protection rules — for example, requiring a specific reviewer to approve before a job in that environment can run.

In this project: The prod environment enforced a required reviewer and wait timer, creating a manual approval gate before any deployment to production.

> **In simple terms:**
> DAB decides WHAT should exist in Databricks. GitHub Actions decides WHEN to check and deploy it. The service principal and OAuth M2M decide WHO (or rather, which automated identity) is allowed to do it.

5. Project Structure

The repository followed the standard DAB layout, extended with a GitHub Actions workflow for CI/CD:

```
DatabricksDAB/
|
|-- databricks.yml           Main bundle configuration
|-- pyproject.toml           Python project & build configuration
|
|-- resources/
|   |-- sample_job.job.yml               Job definition
|   `-- DatabricksDAB_etl.pipeline.yml    Pipeline definition
|
|-- src/
|   |-- DatabricksDAB/
|   |-- DatabricksDAB_etl/
|   `-- sample_notebook.ipynb
|
|-- tests/
|-- fixtures/
|-- .gitignore
|
`-- .github/
    `-- workflows/
        `-- deploy.yml       CI/CD pipeline definition
```

Component	Purpose
databricks.yml	Main entry point for the bundle: name, included resource files, variables, targets (DEV/TEST/PROD), and deployment mode.
resources/	YAML definitions of the Databricks Job and Lakeflow Pipeline that DAB manages.
src/	Application and data-engineering source code (Python modules and the ETL notebook).
pyproject.toml	Declares the Python project and its build configuration.
.github/workflows/deploy.yml	Defines the CI/CD pipeline executed by GitHub Actions.

6. Implementation Details

6.1 Bundle Configuration (databricks.yml)

The bundle configuration declared the project name and included all resource definitions from the resources folder:

```
bundle:
  name: DatabricksDAB
  uuid: 966f2a8c-5c24-46d4-9344-2c83a570bec2

include:
  - resources/*.yml
```

6.2 Python Artifact

The project's Python code is packaged into a wheel file as part of every deployment, using uv:

```
artifacts:
  python_artifact:
    type: whl
    build: uv build --wheel
```

During deployment this produced output such as:

```
Building python_artifact...
Uploading dist/databricksdab-0.0.1-py3-none-any.whl...
```

6.3 Environment Targets — DEV, TEST, PROD

Rather than maintaining three separate codebases, a single bundle was configured with three targets, each supplying environment-specific variable values (catalog, schema) and deployment mode:

```
variables:
  catalog:
    description: The catalog to use
  schema:
    description: The schema to use

targets:
  dev:
    mode: development
    variables:
      catalog: workspace
      schema: ${workspace.current_user.short_name}

  test:
    mode: development

  prod:
    mode: production
    variables:
      catalog: workspace
      schema: prod
```

The deployment target is selected via the -t flag:

```
databricks bundle deploy -t dev     # deploy to DEV
databricks bundle deploy -t test    # deploy to TEST
databricks bundle deploy -t prod    # deploy to PROD
```

6.4 Resource Definitions — Job and Pipeline

The bundle managed two Databricks resources: a Job (sample_job) responsible for executing the workload, and a Lakeflow Pipeline (DatabricksDAB_etl) implementing the ETL process.

```
resources:
  pipelines:
    DatabricksDAB_etl:
      name: ${bundle.target}_DatabricksDAB_etl
      catalog: ${var.catalog}
      schema: ${var.schema}
      serverless: true
      root_path: "../src/DatabricksDAB_etl"

      libraries:
        - glob:
            include: ../src/DatabricksDAB_etl/transformations/**

      environment:
        dependencies:
          - --editable ${workspace.file_path}
```

Naming each resource with ${bundle.target} ensured each environment produced its own uniquely identifiable resource: dev_DatabricksDAB_etl, test_DatabricksDAB_etl, and prod_DatabricksDAB_etl. This resolved an early naming collision (see Section 9).

7. Local Validation and Deployment

Before introducing automation, the bundle was validated and deployed manually to prove the DAB configuration itself worked correctly.

1. Validate the bundle configuration: databricks bundle validate -t test -p free-edition → Validation OK!
2. Deploy to TEST: databricks bundle deploy -t test -p free-edition → Resources: 2 created
3. Run the job: databricks bundle run sample_job -t test -p free-edition → TERMINATED SUCCESS
4. Validate and deploy PROD locally: databricks bundle validate -t prod and databricks bundle deploy -t prod → both succeeded, creating the Job, Pipeline, and their permissions.

This confirmed the chain DAB → Job → Python wheel → Databricks execution was functioning before any CI/CD automation was layered on top.

8. Version Control (Git and GitHub)

The DAB project was initialized as a Git repository and pushed to GitHub, becoming the single source of truth for the project's code and configuration.

```
git init
git add .
git commit -m "Initial DAB project"
git push -u origin main
```

With Git in place, every deployment traces back to a specific, reviewable commit, in contrast to making untracked manual changes directly in the Databricks UI:

```
Code  ->  Git  ->  Review  ->  DAB  ->  Databricks
```

9. CI/CD Pipeline (GitHub Actions)

DAB defines what should exist in Databricks. GitHub Actions decides when to validate or deploy it, by executing Databricks CLI commands on GitHub-hosted runners.

```
GitHub Actions  ->  Databricks CLI  ->  DAB  ->  Databricks APIs  ->  Job / Pipeline
```

9.1 Workflow Triggers

The workflow reacts to two Git events:

```
on:
  pull_request:
    branches:
      - main
  push:
    branches:
      - main
```

On a pull request, only the validate job runs. On a push to main (i.e. after a PR is merged), the full validate → DEV → TEST → PROD sequence runs. This separation is enforced with a condition on every deployment job:

```
if: github.event_name == 'push'
```

9.2 Job Sequencing

Each job declares the job it depends on via needs, forming a strict pipeline:

Job	Depends on / Trigger
validate	Runs on both pull_request and push events.
deploy-dev	needs: validate; if: push
deploy-test	needs: deploy-dev; if: push
deploy-prod	needs: deploy-test; if: push; gated by the prod GitHub Environment

9.3 CI vs. CD in This Project

Phase	What Happens
Continuous Integration (CI)	Pull request → databricks bundle validate -t dev. Confirms the bundle configuration is structurally and semantically valid before any code is merged.
Continuous Deployment (CD)	Merge to main → databricks bundle deploy for DEV, then TEST, then PROD (after approval).

9.4 Authentication — Service Principal and OAuth M2M

GitHub Actions runs on a GitHub-hosted machine with no access to a personal Databricks CLI profile. To authenticate securely and avoid using a personal account, a dedicated Databricks service principal, dab-cicd-poc, was created and used exclusively by the pipeline:

```
GitHub Actions  ->  OAuth M2M  ->  dab-cicd-poc  ->  Databricks
```

Credentials were stored as GitHub Secrets and exposed to the workflow as environment variables:

```
env:
  DATABRICKS_HOST: ${{ secrets.DATABRICKS_HOST }}
  DATABRICKS_CLIENT_ID: ${{ secrets.DATABRICKS_CLIENT_ID }}
  DATABRICKS_CLIENT_SECRET: ${{ secrets.DATABRICKS_CLIENT_SECRET }}
  DATABRICKS_AUTH_TYPE: oauth-m2m
```

9.5 Production Approval Gate

A GitHub Environment named prod was configured with deployment protection rules (a required reviewer and a wait timer). As a result, PROD is never deployed automatically — it requires explicit human approval after TEST succeeds:

```
TEST  ->  PROD Environment  ->  Reviewer approval  ->  Deploy PROD
```

10. Issues Encountered and Resolutions

A number of real-world issues were encountered and resolved during implementation. These are summarized below.

Issue	Cause	Resolution
Duplicate DEV pipeline name (409 RESOURCE_CONFLICT)	An existing pipeline in the workspace already used the auto-generated development name.	Made the pipeline name environment-specific by including the target name (dev/test/prod) as a prefix.
fatal: not a git repository	The project folder had not yet been initialized as a Git repository.	Ran git init, committed, and pushed the project to GitHub.
default auth: cannot configure default credentials	The GitHub-hosted runner had no local Databricks CLI profile.	Configured OAuth M2M via GitHub Secrets and workflow environment variables.
oauth-m2m auth: not configured	Service principal credentials were not yet correctly available to the runner.	Verified and corrected the Databricks authentication environment variables.
uv: command not found	The GitHub Ubuntu runner did not have uv installed (unlike the local machine).	Added the astral-sh/setup-uv@v6 action before the deploy step.
Failed to acquire deployment lock / unable to deploy as dab-cicd-poc	PROD bundle state lived under the personal user's workspace path, inaccessible to the service principal.	Moved the PROD bundle root to /Workspace/Shared/.bundle/Databricks DAB/prod.

Issue	Cause	Resolution
Bundle root path is writable by all workspace users (warning)	/Workspace/Shared is broadly accessible to all workspace users.	Accepted for the POC; flagged as a future hardening step (use a restricted path).
Only admins can change pipeline/job owners	Existing PROD resources were originally owned by a personal account, not the service principal.	Temporarily granted admin access to establish correct ownership, then re-tested without it.
Duplicate PROD pipeline name	An old prod_DatabricksDAB_etl pipeline already existed in the workspace.	Located it with databricks pipelines list-pipelines and deleted the stale resource.
No steps defined in `steps` for deploy-prod	Incorrect YAML indentation / duplicated job properties (environment, runs-on).	Corrected the deploy-prod job structure so environment and steps were properly nested.
Pull requests could trigger full deployment	Deployment jobs had no condition restricting them to push events.	Added if: github.event_name == 'push' to every deployment job.
Service principal had excessive privilege	Admin access had been granted temporarily for troubleshooting.	Removed admin access and re-ran the workflow (Run #12) to confirm it still succeeded.

11. Testing and Validation

The solution was validated in stages rather than assumed to work end-to-end:

Stage	Result
Local TEST — bundle validate	SUCCESS
Local TEST — bundle deploy	SUCCESS
Local TEST — bundle run	SUCCESS (TERMINATED SUCCESS)
GitHub Actions — Deploy DEV	SUCCESS
GitHub Actions — Deploy PROD (service principal with admin)	SUCCESS (Run #11)
GitHub Actions — Deploy PROD (service principal without admin)	SUCCESS (Run #12)
Pull request workflow (feature branch → PR → main)	Validated on PR; deployment jobs correctly skipped; deployment triggered only after merge

The most significant result was Run #12: after removing workspace-admin privileges from the dab-cicd-poc service principal, the full DEV → TEST → PROD pipeline still completed successfully, confirming the deployment identity operates under least-privilege access.

12. Final Architecture

```
                    GitHub
```

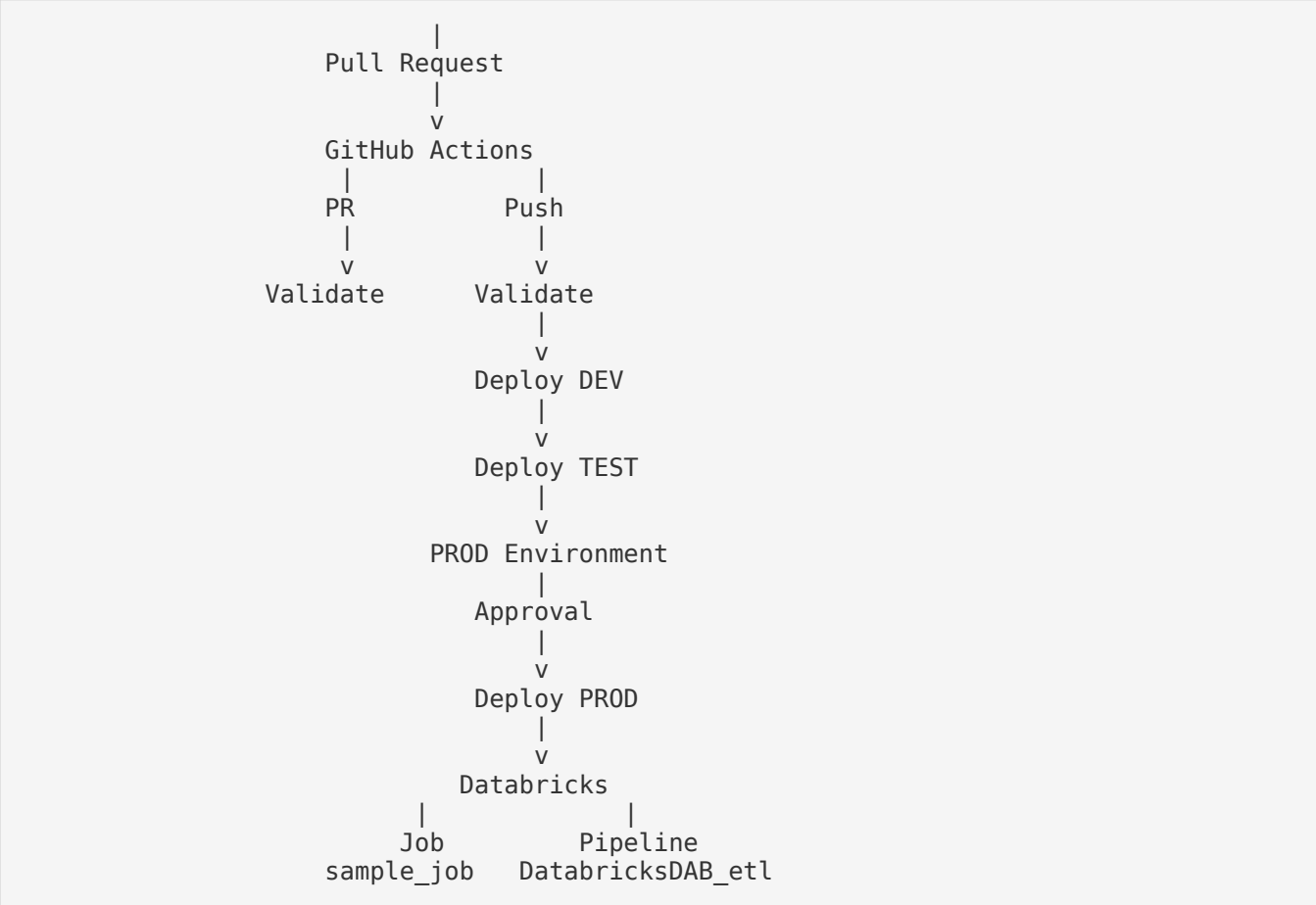

Figure 1: End-to-end deployment flow, from pull request through to production.

Authentication runs underneath every deployment step:

```
GitHub Actions  ->  OAuth M2M  ->  dab-cicd-poc  ->  Databricks
```

13. Appendix A — Final deploy.yml

The complete, final GitHub Actions workflow used to validate and deploy the bundle:

```
name: Databricks DAB CI/CD

on:
  pull_request:
    branches:
      - main
  push:
    branches:
      - main

permissions:
  contents: read

env:
  DATABRICKS_HOST: ${{ secrets.DATABRICKS_HOST }}
```

```
  DATABRICKS_CLIENT_ID: ${{ secrets.DATABRICKS_CLIENT_ID }}
  DATABRICKS_CLIENT_SECRET: ${{ secrets.DATABRICKS_CLIENT_SECRET }}
  DATABRICKS_AUTH_TYPE: oauth-m2m

jobs:

  validate:
    name: Validate DAB
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
      - name: Install Databricks CLI
        uses: databricks/setup-cli@main
      - name: Install uv
        uses: astral-sh/setup-uv@v6
      - name: Validate bundle
        run: databricks bundle validate -t dev

  deploy-dev:
    name: Deploy DEV
    if: github.event_name == 'push'
    needs: validate
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
      - name: Install Databricks CLI
        uses: databricks/setup-cli@main
      - name: Install uv
        uses: astral-sh/setup-uv@v6
      - name: Deploy to DEV
        run: databricks bundle deploy -t dev

  deploy-test:
    name: Deploy TEST
    if: github.event_name == 'push'
    needs: deploy-dev
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
      - name: Install Databricks CLI
        uses: databricks/setup-cli@main
      - name: Install uv
        uses: astral-sh/setup-uv@v6
      - name: Deploy to TEST
        run: databricks bundle deploy -t test

  deploy-prod:
    name: Deploy PROD
    if: github.event_name == 'push'
    needs: deploy-test
    runs-on: ubuntu-latest
    environment:
      name: prod
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
      - name: Install Databricks CLI
```

```
        uses: databricks/setup-cli@main
      - name: Install uv
        uses: astral-sh/setup-uv@v6
      - name: Deploy to PROD
        run: databricks bundle deploy -t prod
```

14. Conclusion

This project implemented a complete CI/CD solution for a Databricks project using Databricks Asset Bundles and GitHub Actions. The Databricks Job and Lakeflow pipeline, together with a Python wheel artifact and environment-specific targets for DEV, TEST, and PROD, were defined entirely as code and stored in GitHub.

A GitHub Actions workflow validates the bundle on every pull request, and automatically promotes it through DEV and TEST after a merge to main, with production protected by a manual approval gate. Authentication is handled by a dedicated Databricks service principal using OAuth M2M, with credentials stored securely as GitHub Secrets.

During implementation, issues involving duplicate resource names, missing CI authentication, a missing build tool on the CI runner, deployment-state permissions, pre-existing resource ownership, and a duplicate PROD resource were each identified and resolved. Finally, admin privileges were removed from the service principal, and the full pipeline was re-verified to succeed under least-privilege access — confirming the deployment automation is both functional and secure by design.

Key Takeaways

- DAB defines what should exist in Databricks — Jobs, Pipelines, artifacts, and permissions — as source-controlled code.
- GitHub Actions defines when validation and deployment happen, driven by pull-request and push events.
- A dedicated service principal with OAuth M2M defines who is authorized to perform deployments, without relying on personal credentials.
- Environment-specific targets (DEV/TEST/PROD) allow one codebase to be deployed consistently and differently per environment.
- A GitHub Environment approval gate ensures production changes require explicit human sign-off.
- The pipeline was proven to work without workspace-admin privileges, satisfying least-privilege security practice.